

Prompt:

Write a Python script showing that AES-CTR alone doesn't protect against tampering or replay named `baseline_ctr.py`. For setup, a sender encrypts a command like "CMD:READ file=report.txt" with AES-CTR. A relay sits in the middle. It can see and change traffic but doesn't have the key. A receiver decrypts and "runs" the command. Keep it in one file with simple functions and readable printed output.

Response: A digital signature is a value that proves a message came from a particular party and wasn't altered. It's the public-key equivalent of a handwritten signature, but it is tied mathematically to the exact message content.

How it works

It uses a key pair: a private key that only the signer holds, and a public key that anyone can have.

Sign: The signer hashes the message and uses their private key to produce a signature over that hash.

Send: The message and signature travel together.

Verify: The receiver uses the signer's public key to check that the signature matches the message. If it does, verification passes. If even one bit of the message changed, it fails.

Only the private key holder can produce a valid signature, but anyone with the public key can check it.

What it gives you

Authenticity: The message came from the holder of the private key.

Integrity: The message wasn't modified in transit.

Non-repudiation: The signer can't plausibly deny signing it, since nobody else has the private key.

It does not provide confidentiality. The message itself is still readable unless you also encrypt it.

Connecting it to your earlier questions

In your tampering demo, a signature over the ciphertext (or the plaintext) would have caught the relay's edits, because the altered packet would fail verification.

In Diffie-Hellman, signatures stop the man-in-the-middle attack. Each side signs its DH public value (A or B), so a relay can't swap in its own.

A signature alone doesn't stop replay. A captured message with a valid signature is still valid when resent, so you still need a nonce, sequence number, or timestamp inside the signed data.

Signatures vs. MACs

A MAC (like HMAC or the tag in AES-GCM) also provides integrity and authenticity, but it uses a shared secret key, so both parties can create it. That's faster, but it means there's no

non-repudiation, since either side could have produced the tag. Signatures use asymmetric keys, so only the signer can produce them, at the cost of more computation.

Common algorithms

Ed25519, ECDSA, and RSA-PSS. Ed25519 is a popular modern choice because it's fast and hard to misuse.

The remaining problem

You need to be sure the public key really belongs to who you think it does. That's what certificates and certificate authorities solve, by having a trusted party sign a statement binding a name to a public key.

Prompt:

What is verify-before-decrypt

Response:

Verify-before-decrypt means the receiver checks the authentication tag (a MAC or signature) on a packet first, and only decrypts if it passes. If verification fails, the packet is discarded without being decrypted or acted on.

Why the order matters

If you decrypt first and check later, you've already run attacker-controlled input through your decryption and parsing code. That opens several problems:

Oracle attacks: If the receiver behaves differently depending on how decryption or parsing fails (different error messages, timing), an attacker can use that as a signal to learn about the plaintext. The classic example is the padding oracle attack on CBC mode, which can recover whole messages without the key.

Wasted work and attack surface: Forged packets get processed by decryption and parsing code that should only ever see authentic data.

Acting on tampered data: In your demo, the receiver decrypted and executed `CMD:WIPE file=secret.txt` because nothing was checked.

Cryptographers sometimes call this the "cryptographic doom principle": if you perform any action on a message before verifying its MAC, you're likely to eventually get into trouble.

How it looks in practice

With separate primitives, use encrypt-then-MAC:

```
Sender: ct = AES-CTR(key_enc, nonce, plaintext)
        tag = HMAC(key_mac, nonce || ct)
```

send nonce || ct || tag

Receiver: recompute tag over nonce || ct

if tag doesn't match (constant-time compare): reject

otherwise: decrypt

Two details matter here. The tag covers the nonce and ciphertext, so the relay can't alter either. And you should use separate keys for encryption and MAC.

AEAD does this for you

AES-GCM and ChaCha20-Poly1305 bundle encryption and authentication. When you call decrypt, they verify the tag internally and raise an error before returning any plaintext. That's why they're recommended over assembling CTR plus a MAC yourself.

What it doesn't cover

Verification proves the packet is authentic and unmodified, but a captured valid packet still verifies when replayed. You still need a sequence number or timestamp inside the authenticated data, checked after verification passes.

If you'd like, I can add a second version of your script (say `fixed_gcm.py`) that uses AES-GCM with a counter, so you can see both attacks get rejected.

Prompt:

What is an AEAD mode?

Response:

AEAD stands for Authenticated Encryption with Associated Data. An AEAD mode is a cipher construction that encrypts a message and authenticates it in a single operation, so you get confidentiality and integrity together instead of assembling them yourself.

What you give it and what you get back

`encrypt(key, nonce, plaintext, associated_data) -> ciphertext || tag`

`decrypt(key, nonce, ciphertext || tag, associated_data) -> plaintext, or an error`

Key and nonce: As before. The nonce must never repeat for a given key.

Plaintext: Encrypted, so it's hidden.

Associated data (AD): Extra data that is authenticated but not encrypted. Think of packet headers, a sequence number, or a protocol version: the receiver needs to read them, but they must not be tampered with.

Tag: A short authentication value (typically 16 bytes) covering the ciphertext and the associated data. Changing any bit of either makes decryption fail.

On decryption, the tag is checked first, and if it doesn't match you get an error and no plaintext. That is verify-before-decrypt built into the primitive.

Common AEAD modes

AES-GCM: The most widely used. It's CTR mode for encryption plus a polynomial-based authenticator (GHASH). It's fast with hardware support, but it needs a unique 12-byte nonce, and reusing one is catastrophic.

ChaCha20-Poly1305: A stream cipher with a Poly1305 MAC. It's fast in software and is used in TLS and WireGuard.

AES-GCM-SIV and XChaCha20-Poly1305: Variants that are more forgiving of nonce mistakes (SIV) or allow safe random nonces (X-variant, 24-byte nonce).

How it relates to your script

Your AES-CTR script used a bare confidentiality mode, which is why the relay could flip bits freely. Swapping in AES-GCM would make the tampered packet fail with an error instead of being decrypted and executed.

AEAD also doesn't stop replay by itself. A captured packet still has a valid tag when resent. The usual fix is to put a sequence number in the associated data (or the plaintext), which is now protected from modification, and have the receiver reject any number it has already seen or that is lower than the last accepted one.

A quick Python example

```
python
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

key = AESGCM.generate_key(bit_length=256)
aead = AESGCM(key)
nonce = os.urandom(12)

ct = aead.encrypt(nonce, b"CMD:READ file=report.txt", b"seq=1")
aead.decrypt(nonce, ct, b"seq=1") # returns the plaintext
aead.decrypt(nonce, ct, b"seq=2") # raises InvalidTag: AD was changed
Flipping any ciphertext bit, or changing the associated data, raises InvalidTag.
```

Prompt: given these requirements - Requirements: • separate keys in each direction; • sequence numbers begin at zero in each direction; • the sender never reuses an IV under one key; • the receiver expects the exact next sequence number; • verify HMAC before decrypting; • reject modified headers, modified ciphertext, replays, and messages in the wrong direction; • do not release plaintext after any error; • use constant-time MAC verification through the library., can you create a python file named secure_record.py that has a seal and open_record function using this format - Use this format: header = version(1 byte) || direction(1 byte) || sequence(8

bytes) || message_type(1 byte) || ciphertext_length(4 bytes) CSCE 465/765 | Computer & Network Security CSCE 465/765 Homework • 4 iv = session_id(8 bytes) || sequence(8 bytes)
ciphertext = AES-256-CTR(K_enc, iv, plaintext) tag = HMAC-SHA-256(K_mac, header || iv || ciphertext). Here is what I have so far - (i cant copy that old file i had)

Response:

Your draft was already correct, so I kept its logic and ran it. All the attack cases are rejected and the normal flow works.

What I changed:

Added a docstring with the record layout and comments on the key steps.

main() now falls back to random test keys if your handshake module isn't importable, so the file runs on its own.

Checked against your requirements:

Per-direction keys and sequence numbers: each direction has its own keys and its own counter starting at 0.

Verify before decrypting: the MAC is checked first and aes_ctr only runs after it passes.

No plaintext on error: every failure raises before decryption.

Constant-time MAC: it uses HMAC.verify() from the library.

State updates: state.seq only advances after a successful seal or open_record, so a bad record can't desync the channel.

Modified length: changing the length field is caught by the length-consistency check, so it shows up as MalformedRecord rather than an MAC failure. That is still a rejection.

A weakness in the format itself: with iv = session_id || sequence, AES-CTR treats the whole 16-byte IV as a counter that increments once per 16-byte block. A message longer than 16 bytes at sequence n therefore uses the counter values that sequence n+1 starts with, which reuses keystream. Your test messages are long enough to trigger this. The IVs are unique as the spec requires, but the counter blocks overlap. I implemented the format exactly as given, since it's the assignment's requirement. I'd mention this in your writeup, and you could ask your instructor about it. A standard fix is a 12-byte nonce with a 4-byte block counter, or putting the sequence number in the high bytes of the IV.